The Tool Command Language (Tcl)

For a visual demonstration of the material in this document, please see the Tcl tutorial at:
- Part 1: https://youtu.be/o_mhSa5HQCc
- Part 2: https://youtu.be/T_4Bvg6np08

1 Introduction

The "Tool Command Language" (TCL or Tcl) is a scripting language, used by the majority of the Electronic Design Automation (EDA) tools intended for chip design. It is a *string-based*, *interpreted* command language.

Note that I have recorded and uploaded a short course on Tcl that you can access through the EnICS Website at: https://enicslabs.com/short-courses/
Or directly access the videos on YouTube:

- Part 1: https://youtu.be/o_mhSa5HQCc
- Part 2: https://youtu.be/T_4Bvg6np08

1.1 Starting Tcl

Basically, you will usually use Tcl from inside your EDA tool shell, such as Innovus or dc_shell. But Tcl is a regular scripting language and a Tcl interpreter comes with any Linux installation and can even be installed on Windows ("He who shall not be named!").

To open a standalone Tcl interpreter, type:

```
%> tclsh
```

1.2 Before basics

Just to understand the examples that come afterwards, let's teach you a few basic commands:

- The "puts" command *puts* a string on the screen (i.e., prints) and ends with a newline, e.g.:

```
% puts "hello, world!"
hello, world!
```

- The "set" command sets the value of a variable, so to create a variable called *a* with the value 5 (i.e., a=5):

```
% set a 5
```

- Variable are evaluated with a $ before their name, so to print the value of *a*:

```
% puts $a
5
```

- The "expr" command evaluates a mathematical expression, so to evaluate 2+2:

```
% expr 2+2
4
```

- Special characters are escaped with a backslash. For example to print "$a" and not the value of the variable a:

```
% puts "\$a"
$a
```

1.3 True basics

- A Tcl script consists of one or more commands, separated by newlines or semicolons.

```
% puts "hello,"; puts "world!"; # writes hello, \n world on screen.
hello,
world!
```

- Lines starting with a # are commented.

Warning: to start a comment in the middle of the line, first end the command with a semicolon, such as in the example above.

- A command is a string followed by zero or more arguments (strings), which are separated by whitespace.

```
% set a 5 ; # set the value of the variable $a to 5
```

- *Polish notation* is used, meaning that internal commands are evaluated first. Internal commands are defined with square brackets [ ].
- Arguments are *quoted by default*. If you want *evaluation* you must ask for it explicitly.

```
% set a 2+2; # the var $a is now the string "2+2"
% set b [expr 2+2]; # the var $b is now 4
% puts "$a = $b"
2+2 = 4
```

1.4 "Everything is a String"

Okay. Remember I wrote above that TCL is a *string-based* language. Well, I'm not sure I emphasized that enough. It *really* is string-based to the point where there is **only one type** in TCL, the string.

This concept may not be that important until you really get into the language, but once you start getting the hang of it, try to think about this point. Everything is a string and these strings get *substituted.* There are three types of *substitutions*:

- Command Substitution

- Variable Substitution
- Backslash Substitution

So basically, as the interpreter reads the code, it sees a string and decides if it is just a string or if it is a command (i.e., procedure/function), a variable (i.e., starts with a `$`) or something special that starts with a backslash (`\`). Just a few points to try and start to perceive this (though, as I said, it doesn't really matter in most cases):

- You don't pass arguments of a certain type to a command. You just pass strings. The procedure will figure out what to do with the argument by itself.
- You can't just write `4+4`, since this is just a string. You can pass `4+4` as an argument to the `eval` command and then it gets interpreted and calculated.

I could write two million additional examples of this type, but I think that's plenty for now.

2 Variables and Lists

2.1 Variable basics

All variables are basically strings, but casting to integer/floating point values is done automatically. There is no need to declare variables or to define (or maintain) their type. The "`set`" command is used to both give a variable a value, as well as to return its value. Within double quotes (`""`) the value of a variable will be evaluated.

```
% set a_string "hello"
% puts "This is a string: $a_string"
This is a string: hello
% set an_int 5
% puts "And this is a number: [expr $an_int + 1]"
And this is a number: 6
% set an_int
5
```

Note that to make sure your variable was understood as a variable, enclose its name (after the dollar sign) with curly braces. For example:

```
% set the_fourth_letter "d"
% puts "abc$the_fourth_letterefgh"
can't read "the_fourth_letterefgh" : no such variable
% puts "abc${the_fourth_letter}efgh"
abcdefgh
```

2.2 List basics

A list is an array of variables (strings). Lists are used all the time and manipulated with many functions. There are three ways to start a list. In all cases, list elements are separated by whitespace:

- Using curly braces:

```
% set list1 {1 2 3 4}
```

- Using double quotes:

```
% set list2 “a b c d”
```

- Using the “list” command:

```
% set list3 [list 1 a b 2 3 c]
```

The big difference between using curly braces `{}` and the quotes or list command is that *variables are not evaluated within curly braces*:

```
% set a 3; set list4 {1 2 $a}; set list5 “1 2 $a”
% puts “list4 is: $list4 \nlist5 is: $list5”
list4 is: 1 2 $a
list5 is: 1 2 3
```

2.3 The foreach command

Maybe the biggest benefit to using a list is iterating through the list with the `foreach` command. This is much *nicer* than using a `for` loop, like in C:

```
% foreach a $list5 {
    puts $a
  }
1
2
3
```

2.4 List functions

There are many functions to manipulate lists, but the most common are:

- `llength`: Find the length of a list:

```
% llength $list5
3
```

- `lindex`: Return the i-th element of a list:

```
% lindex $list5 1
2
```

Note that list indexes start at 0, so the 0-th element of $list5 is 1 and the 2nd element is 3.

And it's really convenient to use the end operator inside `lindex`:

```
% lindex $list5 end
3
% lindex $list5 end-1
2
```

You can (and will) also have *lists inside lists*. In this case, you can find a member as follows:

```
% set mylist { a { b c } }
% puts [lindex $mylist 0 0]
a
% puts [lindex $mylist 1 1]
c
```

- `lappend`: add a value to the end of a list:

```
% lappend list5 4; puts $list5
1 2 3 4
```

- Other useful functions include: `concat, join, linsert, lrange, lreplace, lsearch, lsort, split`

2.5 Arrays

An important, but less known data structure in Tcl is the array. Make sure you don't get mixed up, since what would be considered an *array* in many languages (like C) is more like a *list* in Tcl, while a *Tcl array* is **associative**, and therefore is similar to a *hash* in Perl or *dictionary* in Python (even though Tcl also has dictionaries).

2.5.1 What is an Array

An associative array is a collection of values that can be referred to according to a key.

```
set arrayName(keyName) value
```

For example, I may have an array called `student` with *keys* that are the *first name* of the student and *values* that are the *last names*:

```
% set student(Yossi) "Cohen"
% set student(Michal) "Levi"
% puts $student(Yossi)
Cohen
```

2.5.2 Basic array manipulation

- To find the size (number of key-value pairs) in an array, use the `array size` command:

```
% array size student
2
```

- To retrieve all the keys in the array, use the `array names` command:

```
% array names student
Yossi Michal
```

- Accordingly, to iterate through an array:

```
% foreach first_name [array names student] {
    puts “$first_name $student($first_name)”
  }
Yossi Cohen
Michal Levi
```

2.6 Environment variables

To access UNIX environment variables, use the `env` array:

```
% puts $env(PWD)
/u/engguest/temanad
```

3 Control Flow

Tcl is very similar to other computer languages (C, java, etc.) and scripting languages (Perl, Python, etc.) in regards to control flow. Personally, I almost always use `foreach` and `if` to do things, but you’ve got all the options:

3.1 If-then-else:

`if` *test1 body1* `?elseif` *test2 body2 ...* `?else` *bodyn*

```
% if { $a == 5 } {
     puts “a equals five”
  } elseif { $a == 4 } {
     puts “a equals four”
  } else {
     Puts “a is not four or five”
  }
```

3.2 Foreach:

`foreach` *varName list body*

```
% foreach item “item1 item2 item3” {
     puts “The current item is: $item”
```

```
  }
The current item is: item1
The current item is: item2
The current item is: item3
```

3.3 While:

```
while test body
```

3.4 Switch:

```
switch string
   pattern body
   ?pattern body
   ...
```

4 Procedures

When reusing code, it is very worthwhile to write procedures, which are the Tcl version of functions.

4.1 Procedure declaration

Procedures are declared with the command proc, followed by arguments:

```
proc procedure_name arguments body
```

Usually both arguments and the body are in curly braces. Let's just always write them that way. For example, a procedure to add two numbers:

```
% proc plus { a b } {
     return [expr $a + $b ]
  }
```

To give default values to arguments, put the argument and its default value within another pair of curly braces:

```
% proc plus_v2 { a b {to_print 1} } {
    set c [expr $a+$b]
    if { $to_print } { puts "$a + $b = $c }
    return $c
  }
```

4.2 Calling a procedure

To call a procedure, just write its name followed by arguments, separated by whitespace:

```
% set a [plus 2 2]
% puts $a
4
```

If an argument is omitted, its default value will be used (if it has one):

```
% plus_v2 2 2 1
2 + 2 = 4
% plus_v2 2 2 0
% plus_v2 2 2
2 + 2 = 4
```

4.3 Procedure locality

- Arguments and variables created inside procedures are *local*. They disappear after leaving the procedure.
- The correct way to pass an argument back to the outer script is through the `return` value.

```
% set a [plus 2 2]
% puts $a
4
```

- To change a global variable, define it as global in the procedure:

```
% proc inc_a { } {
     global a ; # use the same $a that exists outside
     set a [incr a] ; increment $a by 1
  }
% set a 1
% inc_a
% puts $a
2
```

Note that there is a much better way to deal with these things. It's called `namespaces`, but I don't want to go into it now. If you really want to be a Tcl power user, read about (and use) namespaces. Cadence does!

5 String Manipulation

5.1 Basics

As mentioned previously, all values in Tcl are strings. Here are a few useful things you can do with them:

```
% if { "this string" == "this string" } {puts "Yes!" }
Yes!
% string index "abcde" 2 ; # return the char at position 2
c
% string length "abcde" ; # find the length of a string
5
% string first "abcdebcdebc" "de" ; # find the first time "de" appears
```

```
3
% string match “*@*.com” “me@me.com” ; # check if the pattern of arg1
              # matches the string of arg2. * matches any character(s)
1
% string match “*@*.com” “me@me.co.il” ; # This shouldn’t match
0
```

5.2 Formatted Output

The `format` command formats a string similar to the C `printf` style of formatting. This is important when, for example, you want to write out a nice text file.

`format “formatting directives” string`

```
% puts [format “%f” 43.5]
43.50000
% puts [format “%e” 43.5]
4.350000e+01
% puts [format “%d\t%s” $a $student(Michal)]
2     Levi
% set long_pi 3.14159265359
% format {%0.2} $long_pi
3.14
```

5.3 Regular Expressions

Regular expressions are one of the most important tools a VLSI engineer can use. The only problem is that I don’t have enough room or time to write much about them here, but I highly advise you to do a Google search and read a bit more about them.

In general, a regular expression (regexp) is a very condensed language for defining string matching patterns. In other words, like the string match command, shown above, but a thousand times more powerful.

- To check if your regular expression matches a string:

`regexp expression string`

- To replace (substitute) parts of a string:

`regsub expression string substitution newVar`

This will replace the matching `expression` in `string` with `substitution` and save the result in `newVar`.

Maybe, one day, I will find the time to give a full tutorial on regular expressions. Until then, know that there is this great tool that all VLSI engineers need to be familiar with and go learn them yourselves!

6 File I/O

File operations are similar to C and most other languages. The basic approach is that you open a file for reading, writing or appending and get a *file handle*, which is a pointer to the current position in the file. After that, all access to the file is through the file handle. Finally, you should close the file, or else you may not have the result you expect.

6.1 Basic File Operations

- Opening a file for reading:

```
% set fh [open "myfile.txt" r]
```

- Opening a file for writing (this will overwrite an existing file!):

```
% set fh [open "myfile.txt" w]
```

- Reading a file

```
% set file_data [read $fh]
```

This will read the entire file into the file_data variable.

- Reading a file line by line – use the `gets` command, which gets a string until a newline.

```
% while { [gets $fh data] >= 0 } {
      puts $data ; # write the file on screen, line by line.
  }
```

- Writing to a file – just use our friendly `puts` command!

```
% puts $fp "Some text"
```

Note that the first argument to the `puts` command is the file pointer to output the second argument (string) to. If it is omitted, then by default, the text is output to the *standard output*, which is just a file pointer to the computer screen!

- Finally, let's close the file

```
% close $fp
```

7 Important Functions

7.1 Math functions

Tcl has built in functions for regular mathematical functions, if they are called within an expr expression. For example:

- `pow x y` – compute x to the power y.
- `cos arg` – compute the cosine of arg.
- `log10 arg` – compute the logarithm of arg.

Pay attention that integers are treated as integers within an expr expression. If you want to treat them as floating point numbers, add a `.0` or cast with `double()`.

```
% expr 1/2
0
% expr 1.0/2
0.5
% expr double(1)/2
0.5
```

7.2 System Functions

- Get the time: `clock`

```
% set currentTime [clock seconds]
1402808400
% clock format $currentTime -format %H:%M:%S
03:09:16
```

- Run a command on the Linux system: `exec`

```
exec ls
exec cd ~
```

As a further expansion of Tcl, you can interface between your EDA tool shells (in Tcl) and an external script (e.g., Python). See the demonstration by Udi Kra at: https://youtu.be/41oU5Tcz4SQ